

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO
FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA
ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



INFORME DE INGENIERÍA DE SOFTWARE
PROCESO DE DESARROLLO Y PRIMER INCREMENTO

“SISTEMA INTEGRAL DE GESTIÓN DE CAPACITACIONES (SIGC-CUSCO)”

Asignatura: Ingeniería de Software I

Docente de Teoría: Prof. Jisbaj Gamarra Salas

Docente de Prácticas: Ing. Lisha Sabah Diaz Caceres

Integrantes:

- | | |
|------------------------------------|--------|
| • Choquenaira Quispe, Noe Franklin | 133962 |
| • Yaranga Achahui, Aldo | 103179 |
| • Ccama Enriquez, Carolay | 210921 |

Cusco – Perú

2026

Índice

1. Datos generales del proyecto	1
1.1. Objetivo general y específicos	1
1.2. Alcance del sistema	1
1.3. Justificación del proyecto	2
2. Contexto del proyecto y restricciones	2
2.1. Tecnologías utilizadas	2
2.2. Stakeholders (Involucrados en el sistema)	3
3. Enfoque de desarrollo seleccionado y justificación	3
3.1. Prácticas adoptadas del proceso	3
4. Roles, responsabilidades y forma de coordinación	4
4.1. Forma de coordinación	4
5. Flujo de trabajo del equipo	4
5.1. Estados del tablero	4
5.2. Límites de Trabajo en Progreso (Límites WIP)	5
5.3. Política de bloqueo	5
5.4. Definición de hecho (Definition of Done - DoD)	5
6. Incrementos presentados del proyecto	5
6.1. Incremento 1: Autenticación RBAC y Gestión de Capacitaciones	6
6.1.1. Funcionalidad desarrollada	6
6.1.2. Arquitectura unificada del servicio de consulta DNI	6
6.1.3. Evidencia de funcionamiento	7
6.1.4. Pruebas o validaciones realizadas	8
6.1.5. Cambios realizados respecto al avance anterior	9
6.1.6. Métricas y seguimiento del proceso	9
6.1.7. Acuerdos de mejora para el siguiente incremento	10
7. Política de uso de IA y registro de uso	10
7.1. Política de uso de IA	10
7.2. Registro de uso de IA (Bitácora)	11
8. Conclusiones	11

1. Datos generales del proyecto

El proyecto **Sistema Integral de Gestión de Capacitaciones (SIGC-CUSCO)** es una plataforma web creada para registrar, controlar y certificar cursos, talleres y capacitaciones de cualquier institución o empresa. Su objetivo es reemplazar el desorden de las hojas de Excel y listas en papel por un sistema centralizado, rápido y confiable.

1.1. Objetivo general y específicos

Objetivo general: Desarrollar una aplicación web para administrar todo el ciclo de las capacitaciones (inscripción, control de asistencia por QR, registro de notas y entrega de certificados digitales con validación pública).

Objetivos específicos:

- Permitir la inscripción en línea validando el DNI y controlando las vacantes disponibles.
- Tomar asistencia de forma rápida en cada clase mediante el escaneo de códigos QR.
- Emitir certificados digitales en PDF con código QR para que cualquiera pueda verificar su validez en la web.
- Generar actas de notas y reportes claros para la administración y auditoría de los cursos.

1.2. Alcance del sistema

Lo que sí incluye el sistema:

- Inicio de sesión con 3 roles: Administrador, Docente y Participante.
- Creación y edición de cursos (fechas, horas, cupos y docente a cargo).
- Matrícula de participantes con DNI.
- Marcado de asistencia con código QR por sesión.
- Registro de calificaciones y descarga de certificados en PDF.
- Portal web público para comprobar si un certificado es auténtico.

Lo que no incluye (Fuera de alcance):

- No incluye pagos con tarjeta en línea (las matrículas se gestionan de forma gratuita o por caja de la institución).
- No es un aula virtual para ver videos o foros (tipo Moodle).
- No realiza pagos de sueldos a los docentes.

1.3. Justificación del proyecto

Actualmente, muchas organizaciones manejan sus cursos con listas impresas y diplomas hechos a mano en Word. Esto provoca errores al escribir nombres, pérdida de registros de asistencia y diplomas fáciles de falsificar. Con SIGC-CUSCO, toda la información se guarda en una sola base de datos y cualquier persona o empresa puede escanear el QR de un certificado para saber si es legítimo en pocos segundos.

2. Contexto del proyecto y restricciones

El proyecto cuenta con un plazo estricto de **dos semanas** de desarrollo (iniciado el 22 de octubre de 2026), trabajado por tres estudiantes de la UNSAAC a tiempo parcial. Para cumplir con el tiempo, se seleccionaron herramientas estables y un flujo de trabajo ágil y ordenado.

2.1. Tecnologías utilizadas

- **Backend: Laravel 13** con **PHP 8.3**. Manejo modular de rutas, controladores, Form Requests tipados, políticas de autorización (RBAC) y el ORM Eloquent sobre MySQL.
- **Base de Datos: MySQL 8.0** (motor InnoDB, cotejamiento `utf8mb4_unicode_ci`). Integridad referencial con claves foráneas e índices únicos para control estricto de cupos y matrículas.
- **Frontend SPA: Inertia.js v3** con **Vue 3 (TypeScript)** y **Tailwind CSS**, ofreciendo una experiencia de aplicación de una sola página reactiva y fluida.
- **Servicios externos de identidad: API REST PeruDevs**, desacoplada mediante la interfaz `DniLookupService` para validación oficial de DNI ante el Registro Nacional de Identificación y Estado Civil (RENIEC).
- **Gestión del proyecto: Jira Software**. Tablero ágil para organizar el backlog de historias (SIGC-XX), limitar el trabajo en curso (WIP) y medir el avance del equipo.
- **Control de versiones y pruebas: GitHub** con flujo de ramas por desarrollador (ramaFRANKI), Pull Requests y suite de pruebas automatizadas con **Pest PHP** y **PHPUnit**.

2.2. Stakeholders (Involucrados en el sistema)

Involucrado	¿Qué hace o qué necesita en el sistema?
Empresas / Instituciones	Convocan a las capacitaciones y revisan los reportes finales de asistencia y aprobados.
Administrador	Crea los cursos, asigna docentes, supervisa las vacantes y autoriza la emisión de certificados.
Docente / Instructor	Dicta las clases, proyecta el código QR para la asistencia y sube las notas de los alumnos.
Participante / Alumno	Se inscribe con su DNI, marca su asistencia escaneando el QR con su celular y descarga su certificado PDF.
Verificador externo	Cualquier persona o empresa que escanea el código QR para comprobar si el certificado es original.

Tabla 1: Principales involucrados en el sistema SIGC-CUSCO.

3. Enfoque de desarrollo seleccionado y justificación

Para un proyecto de solo dos semanas, no es práctico planificar todo al inicio como en el modelo tradicional en cascada. Se eligió un enfoque **adaptativo e incremental**, combinando prácticas sencillas de Scrum, Kanban y XP:

3.1. Prácticas adoptadas del proceso

- **Aportes de Scrum:** Trabajo dividido en ciclos cortos (sprints) con una meta clara de entrega. Se realizan reuniones breves de 10 minutos para revisar el avance diario y una reunión al final del ciclo para ver qué mejorar.
- **Aportes de Kanban:** Uso de un tablero en Jira con columnas visibles y un límite de trabajo en curso (WIP). Esto ayuda a que el equipo no abra muchas tareas a la vez, aplicando la regla de *terminar antes de empezar algo nuevo*.
- **Aportes de XP:** Pruebas automáticas con PHPUnit en las funciones más delicadas (control de cupos y DNI) y revisión obligatoria de código entre compañeros en GitHub antes de unir cambios a la rama principal.
- **Uso de IA en el proceso:** La Inteligencia Artificial se utiliza como asistente para redactar historias de usuario o consultar sintaxis en Laravel. Cada sugerencia de la IA es revisada manualmente y comprobada con pruebas locales antes de usarse.

4. Roles, responsabilidades y forma de coordinación

El equipo está formado por tres estudiantes con funciones claras para no duplicar esfuerzos:

Integrante	Rol principal	Responsabilidades directas
Choquenaira Quispe, Noe Franklin (133962)	Líder Técnico y Desarrollador Full-Stack	Diseña la arquitectura en Laravel 13 con PHP 8.4, programa las rutas, controladores y la seguridad de roles (RBAC). Administra el repositorio en GitHub y aprueba los Pull Requests.
Yaranga Achahui, Aldo (103179)	Base de Datos y Facilitador Jira	Modela las tablas y migraciones en MySQL 8.0, organiza el backlog en Jira y cuida que se respeten los límites de tareas en el tablero.
Ccama Enriquez, Carolay (210921)	Calidad (QA) y Frontend	Diseña las pantallas con Blade y Tailwind CSS, escribe las pruebas automáticas en PHPUnit y revisa que cada tarea cumpla lo prometido.

Tabla 2: Roles y responsabilidades de los integrantes del equipo.

4.1. Forma de coordinación

- **Seguimiento en Jira y GitHub:** Cada cambio se trabaja en una rama de Git ligada a su ticket de Jira. No se inicia ninguna tarea sin tenerla registrada en el tablero.
- **Reunión diaria de 10 minutos:** Revisión rápida del tablero Jira para saber qué se terminó, qué está en desarrollo y si alguien tiene alguna traba técnica.
- **Revisión de código obligatoria:** Ningún integrante puede subir código directamente a la rama principal (main); siempre se requiere la aprobación de otro compañero en GitHub.

5. Flujo de trabajo del equipo

El avance de las tareas se controla con un tablero ágil en **Jira Software**:

5.1. Estados del tablero

1. **Backlog:** Lista de requerimientos pendientes ordenados por prioridad.
2. **To Do (Por hacer):** Tareas elegidas para el sprint que ya tienen criterios de aceptación claros (*Dado / Cuando / Entonces*).

3. **In Progress (En desarrollo):** Tarea en construcción activa dentro de una rama de Git (feature/SIGC-XX).
4. **In Review / QA (En revisión):** Pull Request abierto en GitHub para que otro compañero revise el código y las pruebas.
5. **Done (Hecho):** Tarea terminada, probada y fusionada en la rama principal (main).

5.2. Límites de Trabajo en Progreso (Límites WIP)

Para que el equipo termine tareas antes de abrir otras nuevas, se definieron dos límites fijos:

- **En desarrollo (*In Progress*):** Máximo **2 tareas al mismo tiempo** para los 3 integrantes.
- **En revisión (*In Review*):** Máximo **2 tareas al mismo tiempo**.

Regla del flujo: Si la columna de revisión llega a 2 tareas, se prohíbe jalar tareas nuevas desde *To Do*. Todo el equipo apoya en revisar y aprobar para liberar la columna.

5.3. Política de bloqueo

Si una tarea no puede continuar por un problema técnico o duda del requerimiento:

- Se marca en Jira con bandera roja (Flagged) y se anota el motivo en un comentario.
- La tarea sigue ocupando espacio en el límite WIP para que el problema no se ignore.
- Si el bloqueo dura más de 24 horas, el equipo se reúne para resolverlo de inmediato.

5.4. Definición de hecho (Definition of Done - DoD)

Una tarea solo se pasa a **Done** si cumple lo siguiente:

1. Cumple los criterios de aceptación acordados.
2. Las migraciones de MySQL corren sin fallar (`php artisan migrate:fresh`).
3. Las pruebas automatizadas con PHPUnit pasan en verde.
4. El Pull Request fue revisado y aprobado por otro integrante en GitHub.
5. El cambio está integrado en la rama principal (main) y probado localmente.

6. Incrementos presentados del proyecto

El desarrollo se realiza mediante entregas periódicas de software utilizable. A continuación se reporta el primer incremento concluido:

6.1. Incremento 1: Autenticación RBAC y Gestión de Capacitaciones

El objetivo de este primer ciclo fue montar la base técnica del sistema en Laravel 13, conectar la base de datos MySQL 8.0 y permitir que los administradores gestionen cursos de forma segura.

6.1.1. Funcionalidad desarrollada

- **Estructura base:** Proyecto Laravel 13 con tablas y migraciones vinculadas en MySQL 8.0.
- **Seguridad y roles (RBAC):** Inicio de sesión con permisos diferenciados para Administrador, Docente y Participante mediante políticas (CoursePolicy).
- **Gestión de cursos:** Formulario para crear y editar cursos con código único, título, fechas de inicio/fin, horas académicas, límite de vacantes y docente asignado.
- **Catálogo de cursos:** Interfaz con buscador reactivo y filtros por estado (Abierto, En curso, Concluido, Cancelado).
- **Integración con API RENIEC / PeruDevs:** Servicio desacoplado DniLookupService y controlador REST que consulta en tiempo real nombres y apellidos ante PeruDevs, con caché local de 30 días para evitar latencia y consumo reiterativo.
- **Frontend SPA Reactivo (Inertia.js v3 + Vue 3):** Vistas dinámicas con Tailwind CSS, navegación fluida sin recargas y componente de consulta DNI en tiempo real.

6.1.2. Arquitectura unificada del servicio de consulta DNI

Para garantizar que la verificación de ciudadanos sea reutilizable en todos los módulos (registro, matrícula y emisión de certificados) sin duplicar código, se implementó el patrón de inversión de dependencias:

- **Objeto de Transferencia (DniPerson):** DTO inmutable con tipado estricto que encapsula los datos retornados por la API oficial (DNI, nombres, apellidos paterno/materno, género y código verificador).
- **Contrato (DniLookupService):** Interfaz en app/Contracts/ que desacopla la lógica de negocio del proveedor externo.
- **Implementación con Resiliencia (PeruDevsDniService):** Servicio en app/Services/Dni/ que pre-valida el formato de 8 dígitos, aplica un tiempo de espera controlado (15 segundos) y almacena en la caché de Laravel los resultados válidos por 30 días.
- **Controlador REST y Componente Vue:** Endpoint GET /api/dni/{dni} consumido asíncronamente por el componente DniSearchCard.vue integrado en el catálogo de capacitaciones y el panel de control.

6.1.3. Evidencia de funcionamiento

La plataforma ya funciona en el entorno local, permitiendo iniciar sesión y crear capacitaciones.

Verificación de correo electrónico. Como parte del flujo de autenticación, el sistema envía al usuario un correo con un enlace temporal para confirmar su dirección de correo electrónico. Esta evidencia demuestra que la verificación de cuentas se encuentra integrada en el primer incremento.

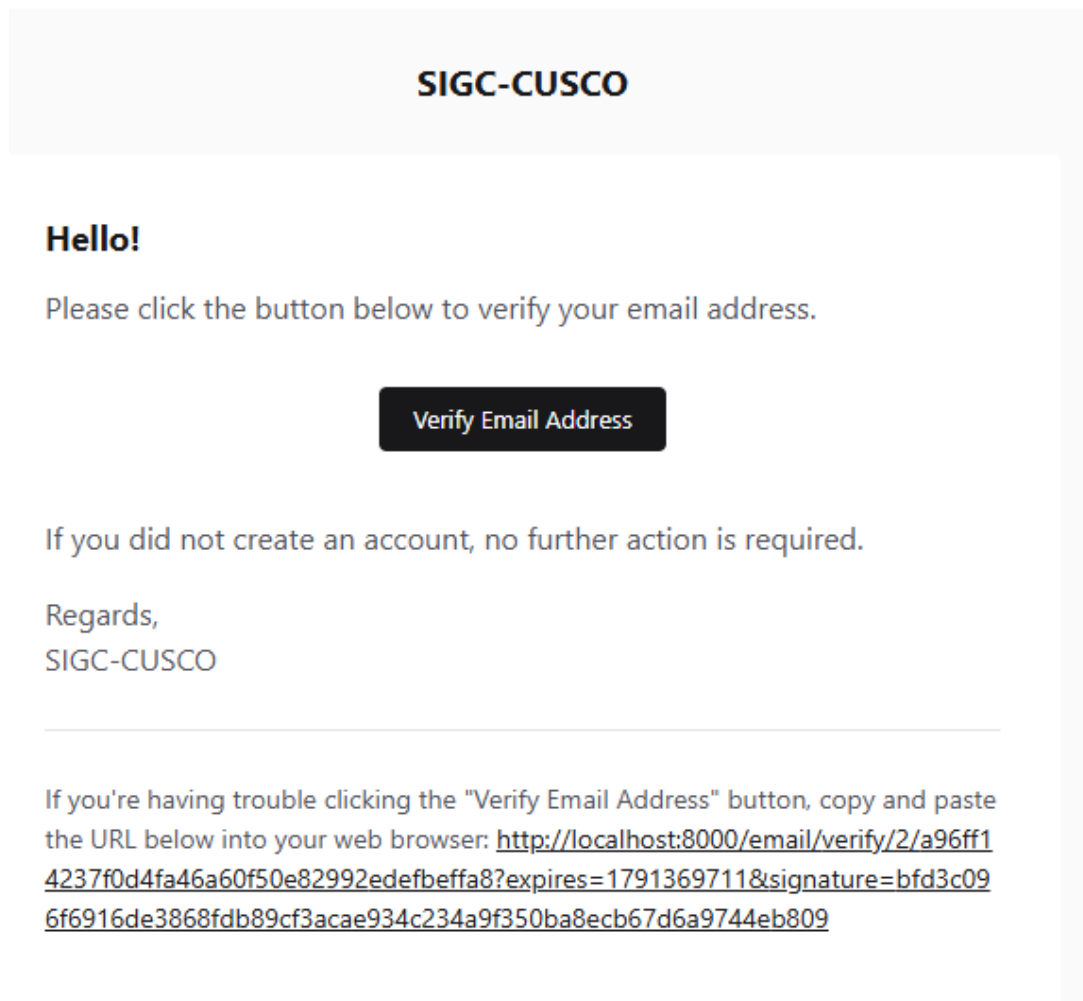


Figura 1: Correo de verificación enviado por SIGC-CUSCO.

En la figura 1 se observa el botón «Verify Email Address» y el enlace alternativo que puede copiarse en el navegador. El enlace se genera de forma temporal para evitar que la confirmación permanezca válida indefinidamente.

[EVIDENCIA PENDIENTE: Capturas de pantalla del sistema]

¿Qué debe ingresar aquí?: Capturas de pantalla de la aplicación web: formulario de login, formulario para crear una capacitación y tabla de cursos guardados en la base de datos. **Ejemplo sugerido:** Pegar una o dos imágenes mostrando la pantalla de login y el listado de capacitaciones activas en <http://localhost:8000/cursos>.

6.1.4. Pruebas o validaciones realizadas

Se implementó una suite integral de pruebas automáticas con Pest PHP y PHPUnit sobre una base de datos MySQL dedicada (SIGC_testing), garantizando el aislamiento del entorno de producción y desarrollo:

Código	Prueba realizada	Resultado esperado	Estado
CP-01	Alumno intenta entrar a la pantalla de crear cursos.	Acceso denegado (Error HTTP 403 vía CoursePolicy).	Aprobado
CP-02	Se intenta guardar un curso con vacantes en 0 o negativo.	Formulario bloquea y muestra mensaje de alerta en español.	Aprobado
CP-03	Se ingresa una fecha de fin menor a la fecha de inicio.	Error de validación por fechas incoherentes (after_or_equal).	Aprobado
CP-04	Se completa el formulario con datos válidos y docente.	Curso persistido en base de datos MySQL y visible en catálogo.	Aprobado
CP-05	Consulta de DNI mediante servicio externo PeruDevs.	Retorna objeto estructurado DniPerson y almacena en caché 30 días.	Aprobado

Tabla 3: Pruebas automáticas ejecutadas en el Incremento 1.

Resultado de ejecución de la suite de pruebas. A continuación se reporta la ejecución de la batería completa de pruebas automatizadas:

```
$ php artisan test
Tests\Feature\Auth\RegistrationTest ..... PASS
Tests\Feature\CourseTest .....
PASS
```

```
[OK] no autenticado o no admin no puede acceder a crear cursos
[OK] capacidad debe ser un entero positivo
[OK] fecha de fin debe ser posterior o igual a fecha de inicio
[OK] admin puede registrar curso valido en mysql
[OK] endpoint dni retorna 422 para dni invalido
Tests\Feature\Services\DniLookupServiceTest ..... PASS
[OK] consulta exitosa retorna DniPerson estructurado
[OK] respuesta exitosa se almacena en cache 30 dias
Tests: 50 passed (166 assertions)
Duration: 2.50s
```

6.1.5. Cambios realizados respecto al avance anterior

- **Uso formal de Jira:** Se abandonaron notas externas para controlar cada tarea con tickets de Jira (SIGC-XX).
- **Tablas ordenadas en MySQL:** Se separaron las tablas de usuarios, roles y cursos para no duplicar datos y asegurar relaciones limpias.
- **Validación estricta en Laravel:** Se configuraron reglas automáticas para impedir que se ingresen cupos negativos o fechas erróneas.

6.1.6. Métricas y seguimiento del proceso

- **Tiempo por tarea (Cycle Time):** En promedio, cada historia tomó 2 días desde que se empezó a programar hasta que se aprobó.
- **Control de tareas en curso:** Se respetó el tope de 2 tareas simultáneas, evitando amontonar trabajo sin terminar.
- **Cumplimiento:** Se terminaron al 100% las dos historias comprometidas (SIGC-1 y SIGC-2).

[EVIDENCIA PENDIENTE: Captura del tablero Jira]

¿Qué debe ingresar aquí?: Captura de pantalla de Jira mostrando las tarjetas en la columna **Done** (Hecho). **Ejemplo sugerido:** Pegar imagen del tablero mostrando los tickets SIGC-1 (Roles) y SIGC-2 (Cursos) en la columna **Done**.

6.1.7. Acuerdos de mejora para el siguiente incremento

1. **Revisión rápida de código:** Atender los Pull Requests en menos de 24 horas para no frenar a los compañeros.
2. **Escribir pruebas antes del código:** Aplicar TDD para la matrícula de alumnos y control de aforo antes de diseñar la pantalla.
3. **Alistar la lectura de QR:** Probar la librería de escaneo con cámara antes de empezar el segundo incremento.

7. Política de uso de IA y registro de uso

En SIGC-CUSCO la Inteligencia Artificial se utiliza como una herramienta de apoyo para consultas técnicas. La autoría y responsabilidad del software es 100 % del equipo de estudiantes.

7.1. Política de uso de IA

- **Uso permitido:** Ayuda para ordenar historias de usuario, consultas puntuales de sintaxis en Laravel 13 y sugerencias de casos de prueba para PHPUnit.
- **Uso prohibido:** Copiar y pegar código sin revisarlo; dejar que la IA decida la estructura del sistema o la base de datos; y subir datos privados o contraseñas a herramientas públicas.
- **Regla obligatoria:** Toda ayuda de la IA debe probarse con PHPUnit y revisarse en equipo antes de integrarla al proyecto.

7.2. Registro de uso de IA (Bitácora)

Fecha / Ticket	Herramienta	¿Qué se consultó?	¿Cómo se revisó y aprobó?
28/09/2026 SIGC-1	Claude 3.5 Sonnet	Mejorar redacción de historias de usuario para roles en formato Gherkin.	Se quitaron supuestos que no pedía el docente y se subieron a Jira. <i>Revisó: N. Choquenaira.</i>
02/10/2026 SIGC-2	ChatGPT (GPT-4o)	Sintaxis de validación en Laravel para que la fecha final no sea antes del inicio.	Se usó la regla nativa de Laravel y se probó con el caso CP-03. <i>Revisó: C. Ccama.</i>
07/10/2026 SIGC-3	Google Antigravity	Diseño de servicio desacoplado y caché para la API REST de DNI PeruDevs.	Se implementó el contrato DniLookupService, DTO inmutable y se validó con suite de pruebas Pest. <i>Revisó: F. Choquenaira.</i>

Tabla 4: Bitácora de consultas técnicas a Inteligencia Artificial.

8. Conclusiones

1. **Organización ágil efectiva:** La combinación de ciclos cortos de Scrum, el tablero con límites de Kanban y las pruebas de XP permitieron al equipo de tres estudiantes coordinarse bien y entregar el primer incremento en el plazo de dos semanas.
2. **Base técnica sólida:** El uso de Laravel 13 junto con MySQL 8.0 aseguró que los datos de usuarios, roles y cursos se guarden de manera ordenada, evitando inconsistencias en los cupos y facilitando la seguridad del sistema.
3. **Disciplina de trabajo en Jira:** Poner un límite de máximo dos tareas a la vez ayudó a que el equipo no se disperse y obligó a revisar y cerrar el código antes de comenzar cosas nuevas.
4. **Uso responsable de la IA:** Utilizar la Inteligencia Artificial solo como apoyo para consultas rápidas ahorró tiempo, manteniendo siempre el control del código mediante pruebas automáticas y revisión manual entre compañeros.